

A zero-knowledge proof of possession of this repository

The Mojo-in-Lean 4 formalization, attested by its own Σ -protocol

Generated from the Lean sources by `scripts/zkpop_pdf.py`

What this document is

The repository contains a formalization of the Mojo language in Lean4, and inside it a zero-knowledge layer: a Schnorr-style Σ -protocol, with completeness, special soundness and perfect honest-verifier zero knowledge proved in Lean, and a bridge from it to claims about Mojo programs.

This document is that layer applied to the repository itself. The prover holds the digests of the 293 Lean sources under `RequestProject/`; it publishes a claim about a `Mojo0` program, a commitment to those digests, and a proof of possession of them. The digests themselves are never part of the published data — and, by the zero-knowledge theorem, could not be recovered from it.

Every number below is emitted from the Lean declarations of `RequestProject/Mojo/ZK/Repository.lean`; every displayed Lean statement is extracted verbatim from the sources. The whole project, including the declarations quoted here, builds with no `sorry` and passes the repository’s own axiom audit: each declaration reaches only `propext`, `Classical.choice` and `Quot.sound`.

1 The published statement at a glance

Quantity	Value
Files measured	293 Lean sources under <code>RequestProject/</code>
Digest base	1 048 573 (prime)
Rolling-hash multiplier	257
Published digest sum	144 731 109
Fuel budget of the attestation	512
Size of the withheld repository code	5 876 bits

The claim that is published is a statement about Mojo code, not about Lean: it says that the `Mojo0` attestation program returns the pair `[293, 144 731 109]` on *some* input. The prover additionally commits to the input it used. A verifier learns the count and the sum; it does not learn which files, in which order, with which digests.

2 The Mojo program that is run

The attestation is a program in `Mojo0`, the minimal bootstrap core of the formalization (`RequestProject/Mojo/Bootstrap/Mini.lean`): top-level recursive functions, one dynamic

value type, a fuel budget bounding call depth. *The program, in Mojo surface syntax:*

```
def count(xs: object) -> object:
  return 0 if xs == [] else 1 + count(__mojo0.tail(xs))

def total(xs: object) -> object:
  return 0 if xs == [] else __mojo0.head(xs) + total(__mojo0.tail(xs))

def attest(xs: object) -> object:
  return __mojo0.cons(count(xs), __mojo0.cons(total(xs), []))
```

...and as the abstract syntax the Lean semantics runs:

```
def attestFn : Func where
  name := "attest"
  params := ["xs"]
  body :=
    .prim .cons
      [.call "count" [.var "xs"],
       .prim .cons [.call "total" [.var "xs"], .lit (.list [])]]

def attestP : Program
```

Its correctness is a theorem about *every* input list and every sufficient fuel budget, proved by induction on the list — not by evaluating one particular input:

```
theorem attest_run (ns : List Int) (k : ℕ) (hk : ns.length + 2 ≤ k) :
  run attestP k "attest" [.list (ns.map .int)]
  = .ok (.list [.int ns.length, .int ns.sum])
```

3 What the prover possesses

The witness is the list of file digests. A digest is a byte-wise rolling hash, started at 1 so that leading zero bytes are visible, reduced modulo the digest base; it is a theorem that a digest is always a residue:

```
def fileDigest (bs : List UInt8) : ℕ

theorem fileDigest_lt (bs : List UInt8) : fileDigest bs < digestBase
```

The list of digests is turned into a single scalar by reading it as a positional numeral in that base, tagged with the number of files. That code is *injective*: it determines the repository's digests exactly.

```
def repoCode (ds : List ℕ) : ℕ

theorem repoCode_injective {ds ds' : List ℕ}
  (hb : ∀ d ∈ ds, d < digestBase) (hb' : ∀ d ∈ ds', d < digestBase)
  (hl : ds.length < digestBase) (hl' : ds'.length < digestBase)
  (h : repoCode ds = repoCode ds') : ds = ds'
```

4 The protocol

Committing is exponentiation in a group of prime order q : the commitment map is an injective homomorphism $\text{gen} : \mathbb{Z}/q \rightarrow G$, $\text{gen}(a + b) = \text{gen}(a) \text{gen}(b)$. Recovering a from $\text{gen}(a)$ is the discrete-logarithm problem in G .

```
structure Commitment (q : ℕ) (G : Type u) [CommGroup G] where
  /-- The commitment map: `gen a` commits to the scalar `a`. -/
  gen : ZMod q → G
  /-- `gen` turns addition of scalars into multiplication in `G`. -/
  gen_add : ∀ a b, gen (a + b) = gen a * gen b
  /-- Distinct scalars have distinct commitments (the scheme is binding). -/
```

```
gen_injective : Function.Injective gen
```

With w the secret scalar, $y = \text{gen}(w)$ the public commitment, r the prover's randomness and c the verifier's challenge, the three moves and the check are

$$a = \text{gen}(r), \quad c \in \mathbb{Z}/q, \quad z = r + cw, \quad \text{accept iff } \text{gen}(z) = a \cdot y^c.$$

The simulator, which sees only y , c and picks z freely, outputs $(\text{gen}(z) (y^c)^{-1}, c, z)$. In the non-interactive (Fiat-Shamir) form the challenge is $c = H(y, a)$ and is not transmitted; the verifier recomputes it.

Applied to the repository, the secret scalar is the repository code read in $\mathbb{Z}/q$, and the published statement is the claim together with the commitment:

```
def repoScalar (q : ℕ) (ds : List ℕ) : ZMod q

def repoCommit (F : Commitment q G) (ds : List ℕ) : G

def repoClaim (ds : List ℕ) (fuel : ℕ) : Claim

def repoStatement (F : Commitment q G) (ds : List ℕ) (fuel : ℕ) : Statement q G

def RepoWellFormed (F : Commitment q G) (S : Statement q G) : Prop
```

5 The four properties, as theorems

Each is an instance of the general theorem of the Σ -protocol; all four are proved, for every repository and every commitment scheme. *Soundness of the certificate format, completeness, knowledge soundness, perfect zero knowledge, binding:*

```
theorem repoClaim_checks (ds : List ℕ) (fuel : ℕ) (hfuel : ds.length + 2 ≤ fuel) :
  (repoClaim ds fuel).checks [.list (digestVals ds)]

theorem repoStatement_verify_prove [NeZero q] (F : Commitment q G) (ds : List ℕ) (fuel : ℕ)
  (r c : ZMod q) :
  (repoStatement F ds fuel).verify F (repoProve F ds r c)

theorem repo_extract_eq_repoScalar [NeZero q] [Fact (Nat.Prime q)] (F : Commitment q G)
  (ds : List ℕ) (fuel : ℕ) {t₁ t₂ : Transcript q G}
  (h₁ : (repoStatement F ds fuel).verify F t₁)
  (h₂ : (repoStatement F ds fuel).verify F t₂)
  (hcom : t₁.commitment = t₂.commitment) (hne : t₁.challenge ≠ t₂.challenge) :
  t₁.extract t₂ = repoScalar q ds

theorem repo_image_prove_eq_image_simulate [NeZero q] [Fintype (ZMod q)] [DecidableEq G]
  (F : Commitment q G) (ds : List ℕ) (fuel : ℕ) (c : ZMod q) :
  (Finset.univ.image fun r => repoProve F ds r c)
  = Finset.univ.image fun z => (repoStatement F ds fuel).simulate F c z

theorem repoCommit_injective_of_lt [NeZero q] (F : Commitment q G) {ds ds' : List ℕ}
  (hb : ∀ d ∈ ds, d < digestBase) (hb' : ∀ d ∈ ds', d < digestBase)
  (hl : ds.length < digestBase) (hl' : ds'.length < digestBase)
  (hq : repoCode ds < q) (hq' : repoCode ds' < q)
  (h : repoCommit F ds = repoCommit F ds') : ds = ds'
```

Read in order: a repository's own digests make the attestation program return the published pair; its holder therefore convinces the verifier for every randomness and every challenge; a prover that answers two different challenges on the same first message hands the extractor the repository's own scalar, so only a holder can do that; the honest conversations, as the randomness ranges over the whole scalar field, are exactly the conversations a simulator writes down from the published statement alone — both maps being injective, so the two uniform distributions coincide — hence the verifier learns nothing about the digests; and the commitment determines the digests as soon as the group order exceeds the code.

6 The proof, for this repository

The measurement, and what it makes the claim say, are themselves kernel-checked statements:

```
def digests : List ℕ

theorem digests_length : digests.length = 293

theorem digests_sum : digests.sum = 144731109

theorem digests_lt : ∀ d ∈ digests, d < digestBase

theorem repoCode_size : 2 ^ 5875 ≤ repoCode digests ∧ repoCode digests < 2 ^ 5876

theorem claim_out : claim = ⟨attestP, "attest", 512, .list [.int 293, .int 144731109]⟩

theorem claim_holds : claim.Holds
```

Two parameter sets are instantiated. Both are safe-prime groups $p = 2q + 1$, with the commitment map $\text{gen}(a) = g^a$ in the subgroup of order q . At the smaller one every step — committing, answering the challenge, verifying — reduces inside the Lean kernel, so the acceptance below is checked by the kernel itself rather than by compiled code.

	q	p	g	y	a	c	z
kernel-checked	11	23	4	9	9	6	1
larger	1 000 151	2 000 303	4	748 910	652 124	830 205	893 402

Here y is the published commitment to the repository, a the prover's first message, $c = H(y, a)$ the Fiat–Shamir challenge and z the response. The verifier's equation $g^z \equiv a y^c \pmod{p}$ holds in both cases:

$$4^1 \equiv 4 \equiv 9 \cdot 9^6 \pmod{23}, \quad 4^{893\,402} \equiv 1\,907\,297 \equiv 652\,124 \cdot 748\,910^{830\,205} \pmod{2\,000\,303}.$$

(That last display is arithmetic recomputed by the document generator. What is authoritative is the Lean side: the verifier's decision procedure run on these very numbers, in the kernel at the smaller parameters and by compiled evaluation at the larger ones.) *The published statement, the published proof, and the verifier accepting it — inside the kernel:*

```
def toyStatement : Statement toyQ ((ZMod toyP)×)

theorem toyStatement_holds : toyStatement.claim.Holds

theorem toyCommitment_val : (toyStatement.commitment : ZMod toyP).val = 9

def toyNIPProof : NIPProof toyQ ((ZMod toyP)×)

theorem toyNIPProof_values :
  ((toyNIPProof.commitment : ZMod toyP).val,
   (Examples.toyHash toyStatement.commitment toyNIPProof.commitment).val,
   toyNIPProof.response.val) = (9, 6, 1)

theorem toyNIPProof_verifies_kernel :
  nizkVerify toyCommitment Examples.toyHash toyStatement.commitment toyNIPProof

theorem toyTranscript_verifies (r c : ZMod toyQ) :
  toyStatement.verify toyCommitment (repoProve toyCommitment digests r c)

theorem toy_extract {t1 t2 : Transcript toyQ ((ZMod toyP)×)}
  (h1 : toyStatement.verify toyCommitment t1) (h2 : toyStatement.verify toyCommitment t2)
  (hcom : t1.commitment = t2.commitment) (hne : t1.challenge ≠ t2.challenge) :
  t1.extract t2 = repoScalar toyQ digests

theorem toy_zeroKnowledge (c : ZMod toyQ) :
  (Finset.univ.image fun r => repoProve toyCommitment digests r c)
```

```
= Finset.univ.image fun z => toyStatement.simulate toyCommitment c z
```

7 What is assumed, and what is not

The parameters are illustrative. The repository code needs 5 876 bits, far more than either group order, so at these parameters the commitment binds the code only *modulo* q ; that is exactly what the following theorem says, and **binding** is the statement that applies once the order is large enough. Nothing else in the development depends on the size of the parameters.

```
theorem toy_binding_residue {ds' : List ℕ}
  (h : repoCommit toyCommitment ds' = toyStatement.commitment) :
  repoCode ds' % toyQ = repoCode digests % toyQ

theorem binding {q : ℕ} [NeZero q] {G : Type*} [CommGroup G] (F : Commitment q G)
  {ds' : List ℕ} (hb' : ∀ d ∈ ds', d < digestBase) (hl' : ds'.length < digestBase)
  (hq : repoCode digests < q) (hq' : repoCode ds' < q)
  (h : repoCommit F ds' = repoCommit F digests) : ds' = digests
```

Hardness is an assumption about the group, not a theorem here. Completeness, special soundness and zero knowledge are unconditional consequences of gen being an injective homomorphism. What makes the protocol worth running — that a non-holder cannot produce an accepting conversation — is the discrete-logarithm assumption in G ; the formalization does not prove it and does not pretend to.

Fiat–Shamir is a modelling assumption. Removing the interaction by setting $c = H(y, a)$ is sound in the random-oracle idealization of H . Every theorem above holds for *every* challenge function H ; the idealization is what turns them into an argument about an adversary.

The digest is a hash, not a proof of file contents. Two different files with the same rolling-hash digest would be indistinguishable to the protocol. What is proved is what the code says: the digest map is injective on its scalar (`repoCode_injective`), and a digest is always a residue below the base (`fileDigest_lt`).

Two files are outside the measurement. The generated manifest `RequestProject/Mojo/ZK/RepoData.lean` and the file that quotes the numbers computed from it, `RequestProject/Mojo/ZK/Repository.lean`, are excluded — their contents are derived from the measurement, so including them would make it refer to itself. The exclusion is recorded in the generated manifest, and the appendix lists exactly what was measured.

The repository is public, so the witness is in it. A real prover would keep the digest list to itself. Here it is committed to the repository as `RepoData.lean`, because a repository that hid it could not be checked by anybody. That is a property of publishing the prover’s state, not of the protocol: the published statement and the published proof reveal nothing about the digests, which is what the zero-knowledge theorem establishes.

8 How to re-check all of this

```
lake build                                # the whole library, incl. the axiom audit
lake env lean scripts/zkpop_manifest.lean # re-measure the repository
lake env lean scripts/zkpop_emit.lean     # re-emit docs/zkpop.json
python3 scripts/zkpop_pdf.py              # re-render this document
```

Re-measuring is idempotent as long as the sources do not change: the manifest generator writes the same file back. Editing any measured file changes the digests, hence the published sum, the commitment and the proof — which is the point of an attestation.

A The files that were measured

The 293 Lean sources whose digests form the witness, in path order. Their digests — the secret — are deliberately not printed.

```
RequestProject/Audit.lean
RequestProject/Automorphic/Groups.lean
RequestProject/Cards/Pixels.lean
RequestProject/Cards/Render.lean
RequestProject/Cards/TheoremCard.lean
RequestProject/Isotropy/Harmonic.lean
RequestProject/Main.lean
RequestProject/MetaMeme/Invariant.lean
RequestProject/Mojo.lean
RequestProject/Mojo/Ast.lean
RequestProject/Mojo/Async.lean
RequestProject/Mojo/Async/Examples.lean
RequestProject/Mojo/Async/Runtime.lean
RequestProject/Mojo/Async/Scheduler.lean
RequestProject/Mojo/Basic.lean
RequestProject/Mojo/Bootstrap.lean
RequestProject/Mojo/Bootstrap/Codegen.lean
RequestProject/Mojo/Bootstrap/CodegenComplete.lean
RequestProject/Mojo/Bootstrap/CodegenExamples.lean
RequestProject/Mojo/Bootstrap/CodegenStmt.lean
RequestProject/Mojo/Bootstrap/
CodegenStmtComplete.lean
RequestProject/Mojo/Bootstrap/
CodegenStmtExamples.lean
RequestProject/Mojo/Bootstrap/Compile.lean
RequestProject/Mojo/Bootstrap/CompileExamples.lean
RequestProject/Mojo/Bootstrap/Core.lean
RequestProject/Mojo/Bootstrap/CoreDyn.lean
RequestProject/Mojo/Bootstrap/CoreDynCompile.lean
RequestProject/Mojo/Bootstrap/CoreDynExamples.lean
RequestProject/Mojo/Bootstrap/Elab.lean
RequestProject/Mojo/Bootstrap/ElabFor.lean
RequestProject/Mojo/Bootstrap/ElabForExamples.lean
RequestProject/Mojo/Bootstrap/ElabLoopElse.lean
RequestProject/Mojo/Bootstrap/ElabParamFor.lean
RequestProject/Mojo/Bootstrap/ElabTryElse.lean
RequestProject/Mojo/Bootstrap/ElabTryFinally.lean
RequestProject/Mojo/Bootstrap/
ElabTryFinallyCheck.lean
RequestProject/Mojo/Bootstrap/
ElabTryFinallyExamples.lean
RequestProject/Mojo/Bootstrap/ElabUnpack.lean
RequestProject/Mojo/Bootstrap/ElabWith.lean
RequestProject/Mojo/Bootstrap/ElabWithBase.lean
RequestProject/Mojo/Bootstrap/ElabWithCheck.lean
RequestProject/Mojo/Bootstrap/ElabWithCheckMain.lean
RequestProject/Mojo/Bootstrap/ElabWithExamples.lean
RequestProject/Mojo/Bootstrap/ElabWithMain.lean
RequestProject/Mojo/Bootstrap/Eval.lean
RequestProject/Mojo/Bootstrap/EvalCongr.lean
RequestProject/Mojo/Bootstrap/EvalMono.lean
RequestProject/Mojo/Bootstrap/Examples.lean
RequestProject/Mojo/Bootstrap/Fixpoint.lean
RequestProject/Mojo/Bootstrap/Lexer.lean
RequestProject/Mojo/Bootstrap/Mini.lean
RequestProject/Mojo/Bootstrap/Mini1.lean
RequestProject/Mojo/Bootstrap/Mini1Compile.lean
RequestProject/Mojo/Bootstrap/Mini1CompileHost.lean
RequestProject/Mojo/Bootstrap/
Mini1CompileHostBridge.lean
RequestProject/Mojo/Bootstrap/
Mini1CompileHostExamples.lean
RequestProject/Mojo/Bootstrap/
Mini1CompileHostProof.lean
RequestProject/Mojo/Bootstrap/Mini2.lean
RequestProject/Mojo/Bootstrap/Mini2Decl.lean
RequestProject/Mojo/Bootstrap/Mini2Examples.lean
RequestProject/Mojo/Bootstrap/Mini2FromSource.lean
RequestProject/Mojo/Bootstrap/Mini2FrontEndHost.lean
RequestProject/Mojo/Bootstrap/Mini2Host.lean
RequestProject/Mojo/Bootstrap/Mini2Inherit.lean
RequestProject/Mojo/Bootstrap/
Mini2InheritExamples.lean
RequestProject/Mojo/Bootstrap/Mini2Lower.lean
RequestProject/Mojo/Bootstrap/Mini2LowerHost.lean
RequestProject/Mojo/Bootstrap/
Mini2LowerHostBridge.lean
RequestProject/Mojo/Bootstrap/
Mini2LowerHostExamples.lean
RequestProject/Mojo/Bootstrap/Mini2Machine.lean
RequestProject/Mojo/Bootstrap/Mini2Mono.lean
RequestProject/Mojo/Bootstrap/Mini2MonoExamples.lean
RequestProject/Mojo/Bootstrap/Mini2MonoHost.lean
RequestProject/Mojo/Bootstrap/
Mini2MonoHostBridge.lean
RequestProject/Mojo/Bootstrap/
Mini2MonoHostExamples.lean
RequestProject/Mojo/Bootstrap/Mini2MonoKey.lean
RequestProject/Mojo/Bootstrap/
Mini2MonoStructExamples.lean
RequestProject/Mojo/Bootstrap/Mini2MonoSubst.lean
RequestProject/Mojo/Bootstrap/Mini2Param.lean
RequestProject/Mojo/Bootstrap/Mini2ParamExamples.lean
RequestProject/Mojo/Bootstrap/Mini2ParamStruct.lean
RequestProject/Mojo/Bootstrap/
Mini2ParamStructExamples.lean
RequestProject/Mojo/Bootstrap/Mini2Parse.lean
RequestProject/Mojo/Bootstrap/Mini2ParseCode.lean
RequestProject/Mojo/Bootstrap/Mini2ParseCodeDecl.lean
RequestProject/Mojo/Bootstrap/Mini2ParseCodeExpr.lean
RequestProject/Mojo/Bootstrap/
Mini2ParseCodeExprBridge.lean
RequestProject/Mojo/Bootstrap/
Mini2ParseCodeFrontEnd.lean
RequestProject/Mojo/Bootstrap/Mini2ParseCodeHost.lean
RequestProject/Mojo/Bootstrap/
Mini2ParseCodeHostDecl.lean
RequestProject/Mojo/Bootstrap/
Mini2ParseCodeHostDeclProg.lean
RequestProject/Mojo/Bootstrap/
Mini2ParseCodeHostDeclProof.lean
RequestProject/Mojo/Bootstrap/
Mini2ParseCodeHostExpr.lean
RequestProject/Mojo/Bootstrap/
Mini2ParseCodeHostExprProof.lean
RequestProject/Mojo/Bootstrap/
Mini2ParseCodeHostProgProof.lean
RequestProject/Mojo/Bootstrap/
Mini2ParseCodeHostStructProof.lean
RequestProject/Mojo/Bootstrap/Mini2ParseExamples.lean
RequestProject/Mojo/Bootstrap/Mini2PlusFrontEnd.lean
RequestProject/Mojo/Bootstrap/Mini2PlusPrinter.lean
RequestProject/Mojo/Bootstrap/Mini2Print.lean
RequestProject/Mojo/Bootstrap/Mini2PrintCode.lean
RequestProject/Mojo/Bootstrap/Mini2PrintHost.lean
RequestProject/Mojo/Bootstrap/Mini2PrintHostDecl.lean
RequestProject/Mojo/Bootstrap/
Mini2PrintHostExamples.lean
RequestProject/Mojo/Bootstrap/Mini2PrintHostExpr.lean
RequestProject/Mojo/Bootstrap/Mini2PrintHostVal.lean
RequestProject/Mojo/Bootstrap/Mini2Surface.lean
RequestProject/Mojo/Bootstrap/
Mini2SurfaceExamples.lean
RequestProject/Mojo/Bootstrap/MiniCodegen.lean
RequestProject/Mojo/Bootstrap/
MiniCodegenExamples.lean
```

RequestProject/Mojo/Bootstrap/MiniCodegenHost.lean
 RequestProject/Mojo/Bootstrap/
 MiniCodegenPipeline.lean
 RequestProject/Mojo/Bootstrap/
 MiniCompilerFixpoint.lean
 RequestProject/Mojo/Bootstrap/MiniCompilerHost.lean
 RequestProject/Mojo/Bootstrap/
 MiniCompilerHostExamples.lean
 RequestProject/Mojo/Bootstrap/MiniExamples.lean
 RequestProject/Mojo/Bootstrap/MiniFrontEnd.lean
 RequestProject/Mojo/Bootstrap/MiniFrontEndBridge.lean
 RequestProject/Mojo/Bootstrap/MiniLexAll.lean
 RequestProject/Mojo/Bootstrap/MiniLexBridge.lean
 RequestProject/Mojo/Bootstrap/MiniLexNumeral.lean
 RequestProject/Mojo/Bootstrap/MiniLexParts.lean
 RequestProject/Mojo/Bootstrap/MiniLexWord.lean
 RequestProject/Mojo/Bootstrap/MiniMinimal.lean
 RequestProject/Mojo/Bootstrap/MiniParseDecl.lean
 RequestProject/Mojo/Bootstrap/
 MiniParseDeclBridge.lean
 RequestProject/Mojo/Bootstrap/MiniParseDeclProof.lean
 RequestProject/Mojo/Bootstrap/MiniParseExpr.lean
 RequestProject/Mojo/Bootstrap/
 MiniParseExprBridge.lean
 RequestProject/Mojo/Bootstrap/MiniParseExprProof.lean
 RequestProject/Mojo/Bootstrap/MiniParseVal.lean
 RequestProject/Mojo/Bootstrap/MiniPlus.lean
 RequestProject/Mojo/Bootstrap/MiniPlusExamples.lean
 RequestProject/Mojo/Bootstrap/MiniPlusFixpoint.lean
 RequestProject/Mojo/Bootstrap/MiniPlusFrontEnd.lean
 RequestProject/Mojo/Bootstrap/MiniPlusLift.lean
 RequestProject/Mojo/Bootstrap/MiniPlusOps.lean
 RequestProject/Mojo/Bootstrap/MiniPrintDecl.lean
 RequestProject/Mojo/Bootstrap/MiniPrintExpr.lean
 RequestProject/Mojo/Bootstrap/MiniPrintNum.lean
 RequestProject/Mojo/Bootstrap/MiniPrintNumBridge.lean
 RequestProject/Mojo/Bootstrap/MiniPrintToks.lean
 RequestProject/Mojo/Bootstrap/MiniPrintVal.lean
 RequestProject/Mojo/Bootstrap/MiniStrings.lean
 RequestProject/Mojo/Bootstrap/MiniSurface.lean
 RequestProject/Mojo/Bootstrap/Parser.lean
 RequestProject/Mojo/Bootstrap/ParserExamples.lean
 RequestProject/Mojo/Bootstrap/Pipeline.lean
 RequestProject/Mojo/Bootstrap/SelfHost.lean
 RequestProject/Mojo/Bootstrap/SelfHostBasic.lean
 RequestProject/Mojo/Bootstrap/SelfHostFuncs.lean
 RequestProject/Mojo/Bootstrap/SelfHostProof.lean
 RequestProject/Mojo/Bootstrap/Soundness.lean
 RequestProject/Mojo/Calls.lean
 RequestProject/Mojo/Complete.lean
 RequestProject/Mojo/Complete/Ast.lean
 RequestProject/Mojo/Complete/Async.lean
 RequestProject/Mojo/Complete/Calls.lean
 RequestProject/Mojo/Complete/Core.lean
 RequestProject/Mojo/Complete/Elab.lean
 RequestProject/Mojo/Complete/Errors.lean
 RequestProject/Mojo/Complete/GPU.lean
 RequestProject/Mojo/Complete/Interop.lean
 RequestProject/Mojo/Complete/Lexical.lean
 RequestProject/Mojo/Complete/Lifecycle.lean
 RequestProject/Mojo/Complete/Modules.lean
 RequestProject/Mojo/Complete/Origins.lean
 RequestProject/Mojo/Complete/Overloads.lean
 RequestProject/Mojo/Complete/Ownership.lean
 RequestProject/Mojo/Complete/Parameters.lean
 RequestProject/Mojo/Complete/Stdlib.lean
 RequestProject/Mojo/Complete/Traits.lean
 RequestProject/Mojo/Errors.lean
 RequestProject/Mojo/GPU.lean
 RequestProject/Mojo/GPU/Atomic.lean
 RequestProject/Mojo/GPU/Block.lean
 RequestProject/Mojo/GPU/Examples.lean
 RequestProject/Mojo/GPU/Kernel.lean
 RequestProject/Mojo/GPU/Reduce.lean
 RequestProject/Mojo/GPU/Warp.lean
 RequestProject/Mojo/Interop.lean
 RequestProject/Mojo/Interop/Arc.lean
 RequestProject/Mojo/Interop/CABI.lean
 RequestProject/Mojo/Interop/Examples.lean
 RequestProject/Mojo/Interop/MemMove.lean
 RequestProject/Mojo/Interop/Memory.lean
 RequestProject/Mojo/Interop/Move.lean
 RequestProject/Mojo/Layout.lean
 RequestProject/Mojo/Lifecycle.lean
 RequestProject/Mojo/Literals.lean
 RequestProject/Mojo/Origins.lean
 RequestProject/Mojo/Origins/Elab.lean
 RequestProject/Mojo/Origins/Examples.lean
 RequestProject/Mojo/Origins/Frame.lean
 RequestProject/Mojo/Origins/Functions.lean
 RequestProject/Mojo/Origins/FunctionsExamples.lean
 RequestProject/Mojo/Origins/Invariants.lean
 RequestProject/Mojo/Origins/Safety.lean
 RequestProject/Mojo/Origins/Semantics.lean
 RequestProject/Mojo/Origins/Syntax.lean
 RequestProject/Mojo/Overloads.lean
 RequestProject/Mojo/OverloadsCall.lean
 RequestProject/Mojo/OverloadsCallSurface.lean
 RequestProject/Mojo/Ownership.lean
 RequestProject/Mojo/Ownership/Elab.lean
 RequestProject/Mojo/Ownership/Examples.lean
 RequestProject/Mojo/Ownership/Safety.lean
 RequestProject/Mojo/Ownership/Semantics.lean
 RequestProject/Mojo/Ownership/Syntax.lean
 RequestProject/Mojo/Parameters.lean
 RequestProject/Mojo/Parameters/Elab.lean
 RequestProject/Mojo/Parameters/Examples.lean
 RequestProject/Mojo/Parameters/Mono.lean
 RequestProject/Mojo/Parameters/Semantics.lean
 RequestProject/Mojo/Parameters/Specialize.lean
 RequestProject/Mojo/Parameters/Syntax.lean
 RequestProject/Mojo/Parameters/Typing.lean
 RequestProject/Mojo/Project.lean
 RequestProject/Mojo/RangeFor.lean
 RequestProject/Mojo/SIMD.lean
 RequestProject/Mojo/Status.lean
 RequestProject/Mojo/Stdlib.lean
 RequestProject/Mojo/Stdlib/Algorithm.lean
 RequestProject/Mojo/Stdlib/Atomic.lean
 RequestProject/Mojo/Stdlib/Base64.lean
 RequestProject/Mojo/Stdlib/BitReverse.lean
 RequestProject/Mojo/Stdlib/BitSet.lean
 RequestProject/Mojo/Stdlib/Bits.lean
 RequestProject/Mojo/Stdlib/ComplexModule.lean
 RequestProject/Mojo/Stdlib/Counter.lean
 RequestProject/Mojo/Stdlib/Deque.lean
 RequestProject/Mojo/Stdlib/DequeBounded.lean
 RequestProject/Mojo/Stdlib/DequeRotate.lean
 RequestProject/Mojo/Stdlib/Dict.lean
 RequestProject/Mojo/Stdlib/DictApi.lean
 RequestProject/Mojo/Stdlib/DictExamples.lean
 RequestProject/Mojo/Stdlib/DictGrow.lean
 RequestProject/Mojo/Stdlib/DictSet.lean
 RequestProject/Mojo/Stdlib/DictTotal.lean
 RequestProject/Mojo/Stdlib/FloatAssoc.lean
 RequestProject/Mojo/Stdlib/FloatDecimal.lean
 RequestProject/Mojo/Stdlib/FloatSemantics.lean
 RequestProject/Mojo/Stdlib/FloatSort.lean
 RequestProject/Mojo/Stdlib/Format.lean
 RequestProject/Mojo/Stdlib/FormatWriter.lean
 RequestProject/Mojo/Stdlib/Hash.lean
 RequestProject/Mojo/Stdlib/IntervalTree.lean
 RequestProject/Mojo/Stdlib/LinkedList.lean
 RequestProject/Mojo/Stdlib/ListBuf.lean
 RequestProject/Mojo/Stdlib/ListSlice.lean
 RequestProject/Mojo/Stdlib/MathModule.lean
 RequestProject/Mojo/Stdlib/MathNumeric.lean
 RequestProject/Mojo/Stdlib/MojoSet.lean

RequestProject/Mojo/Stdlib/MojoString.lean	RequestProject/Mojo/Stdlib/SpanSlice.lean
RequestProject/Mojo/Stdlib/MojoStringCase.lean	RequestProject/Mojo/Stdlib/StringConv.lean
RequestProject/Mojo/Stdlib/MojoStringFind.lean	RequestProject/Mojo/Stdlib/StringOrder.lean
RequestProject/Mojo/Stdlib/MojoStringPartition.lean	RequestProject/Mojo/Stdlib/Testing.lean
RequestProject/Mojo/Stdlib/MojoStringSplit.lean	RequestProject/Mojo/Stdlib/Variant.lean
RequestProject/Mojo/Stdlib/MojoStringSplitStr.lean	RequestProject/Mojo/Stdlib/Writer.lean
RequestProject/Mojo/Stdlib/MojoStringUtf8.lean	RequestProject/Mojo/Swizzle.lean
RequestProject/Mojo/Stdlib/Optional.lean	RequestProject/Mojo/Traits.lean
RequestProject/Mojo/Stdlib/PathCommon.lean	RequestProject/Mojo/Traits/Elab.lean
RequestProject/Mojo/Stdlib/PathModule.lean	RequestProject/Mojo/Traits/Refine.lean
RequestProject/Mojo/Stdlib/PathRelative.lean	RequestProject/Mojo/ZK.lean
RequestProject/Mojo/Stdlib/Random.lean	RequestProject/Mojo/ZK/Attest.lean
RequestProject/Mojo/Stdlib/Range.lean	RequestProject/Mojo/ZK/Encode.lean
RequestProject/Mojo/Stdlib/Reduction.lean	RequestProject/Mojo/ZK/Examples.lean
RequestProject/Mojo/Stdlib/ShuffleUniform.lean	RequestProject/Mojo/ZK/Generator.lean
RequestProject/Mojo/Stdlib/SliceIndices.lean	RequestProject/Mojo/ZK/Instances.lean
RequestProject/Mojo/Stdlib/Sort.lean	RequestProject/Mojo/ZK/Or.lean
RequestProject/Mojo/Stdlib/SortPartition.lean	RequestProject/Mojo/ZK/ProofOfProof.lean
RequestProject/Mojo/Stdlib/SortStable.lean	RequestProject/Mojo/ZK/Sigma.lean
RequestProject/Mojo/Stdlib/Span.lean	RequestProject/Mojo/ZK/Statement.lean